Звіт з лабораторної роботи №5

Тема: Locally finite algorithms

Мета роботи: Реалізувати багатопотокові алгоритми для роботи з графами (пошук найкоротшого шляху та локально оптимальне розфарбування), гарантуючи відсутність стану гонитви (data race) та взаємних блокувань (deadlocks).

Виконав: студент групи ММШІ-1 Красіков Антон

Завдання 1. Алгоритм найкоротшого шляху (SP problem)

1. Постановка задачі

Задається орієнтований граф та одна його стартова вершина. Необхідно для всіх інших вершин знайти мінімальну відстань до заданої вершини за N = |V| ітерацій. Алгоритм повинен бути розпаралелений за допомогою потоків з гарантією відсутності data race при конкурентному доступі до масиву відстаней.

2. Спосіб розв’язання

Для уникнення data race без використання ресурсомістких м'ютексів був застосований архітектурний патерн Double Buffering у поєднанні з розподілом даних.

Замість того, щоб потоки одночасно читали та писали в один масив, створено два масиви. На парних ітераціях потоки читають дані з першого масиву і пишуть у другий, а на непарних - навпаки. Кожен потік записує інформацію виключно у свій, заздалегідь визначений діапазон вершин, що структурно унеможливлює конфлікти при записі.

3. Деталі реалізації

- **Структура графа:** Для оптимізації алгоритму граф зберігається у вигляді списку вхідних ребер. Це дозволяє вершині миттєво отримувати доступ до сусідів, від яких до неї можна прийти, не обходячи весь граф.

- **Синхронізація:** Для забезпечення коректного переходу між ітераціями використовується примітив синхронізації `std::barrier` (стандарт C++20). Він блокує швидкі потоки наприкінці ітерації, поки всі інші не завершать обробку своїх сегментів масиву, після чого всі потоки одночасно переходять до наступного кроку.
- **Відсутність Data Race:** Запис відбувається у `D[next_idx]`, а читання з `D[current_idx]`. Жоден потік не перетинається по індексах запису з іншим потоком.

4. Результати та код

Після запуску програми з 4 потоками для графа з 10 вершинами було отримано наступні результати:

Microsoft Visual Studio Debug Console

```
Initializing Graph Generation...

Generated Graph Structure (Incoming Edges)
Vertex  0 receives from: (v4, w=3)
Vertex  1 receives from: (v0, w=4) (v4, w=1) (v0, w=10)
Vertex  2 receives from: (v1, w=8)
Vertex  3 receives from: (v2, w=10)
Vertex  4 receives from: (v3, w=2) (v0, w=9)
Vertex  5 receives from: (v4, w=8) (v6, w=1)
Vertex  6 receives from: (v5, w=8) (v3, w=2) (v7, w=1) (v1, w=4) (v5, w=4)
Vertex  7 receives from: (v6, w=6)
Vertex  8 receives from: (v7, w=6)
Vertex  9 receives from: (v8, w=2) (v0, w=8) (v8, w=3)


Starting parallel Shortest Path algorithm...
Working with 4 threads.

STATISTICS:
Total Vertices (V) : 10
Total Edges (E)    : 20
Worker Threads     : 4
Target Iterations  : 10 (Formula N = |V|)
Total Barrier Syncs: 10

Minimal Distances from Vertex 0
Vertex  0 : 0
Vertex  1 : 4
Vertex  2 : 12
Vertex  3 : 22
Vertex  4 : 9
Vertex  5 : 9
Vertex  6 : 8
Vertex  7 : 14
Vertex  8 : 20
Vertex  9 : 8
```

Код програми на C++ (папка lab5/lab5_1):

`https://github.com/chesswondo/distributed-systems-labs`

Завдання 2. Локально оптимальне розфарбування графа

1. Постановка задачі

Задається неорієнтований граф. Необхідно провести процедуру мінімізації розфарбування задля знаходження локально оптимального розфарбування (жадібний підхід). Жодні дві суміжні вершини не можуть мати однаковий колір. Алгоритм полягає у випадковому виборі вершини та присвоєнні їй найменшого доступного кольору. Необхідно забезпечити потокобезпечність та вивести кількість ітерацій.

2. Спосіб розв’язання

Оскільки потоки випадковим чином обирають вершини, використання подвійної буферизації тут неможливе. Виникає ризик як data race (одночасне фарбування суміжних вершин), так і взаємних блокувань.

Проблема вирішується за допомогою механізму точкових блокувань із суворим дотриманням патерну Lock Hierarchy. Щоб уникнути deadlocks, потік збирає ID цільової вершини та всіх її сусідів, сортує їх за зростанням, і лише в такому порядку захоплює м'ютекси. Це математично гарантує неможливість циклічного очікування ресурсів.

3. Деталі реалізації

- **Блокування (Mutexes):** Створено масив розумних вказівників на м'ютекси `std::unique_ptr<std::mutex>` (по одному на кожну вершину).
- **Мінімізація (MEX - Minimum Excluded):** Знаходячись у критичній секції (коли вершина та її сусіди заблоковані), потік збирає кольори сусідів у `std::set` і знаходить найменший невід'ємний цілий колір, якого немає серед сусідів.
- **Умова виходу:** Оскільки алгоритм асинхронний і випадковий, для визначення «стабілізації» графа введено глобальний атомарний лічильник `std::atomic<int> idle_attempts`. Якщо протягом великої кількості спроб (10*|V|) жоден потік не зміг змінити колір жодної вершини, граф вважається оптимально розфарбованим, і потоки завершують роботу.

4. Результати та код

Після запуску паралельного алгоритму розфарбування для неорієнтованого графа було успішно знайдено локальний оптимум. Валідація підтвердила відсутність конфліктів кольорів:

```
Generated Undirected Graph:
Vertex  0 connects to: 1 6 3 7
Vertex  1 connects to: 0 2
Vertex  2 connects to: 1 3 14 6 9
Vertex  3 connects to: 2 4 0
Vertex  4 connects to: 3 5 9
Vertex  5 connects to: 4 6 11 12
Vertex  6 connects to: 5 7 2 0
Vertex  7 connects to: 6 8 0
Vertex  8 connects to: 7 9
Vertex  9 connects to: 8 10 2 4
Vertex 10 connects to: 9 11 14
Vertex 11 connects to: 10 12 5
Vertex 12 connects to: 11 13 5 14
Vertex 13 connects to: 12 14
Vertex 14 connects to: 13 2 10 12


Starting Parallel Local Minimization Coloring with 4 threads...

STATISTICS:
Total Vertices (V)       : 15
Total Edges (E)          : 25
Worker Threads           : 4
Total Attempts           : 187
Successful Color Changes : 15


Final Locally Optimal Coloring:
Vertex  0 -> Color 1
Vertex  1 -> Color 0
Vertex  2 -> Color 1
Vertex  3 -> Color 0
Vertex  4 -> Color 1
Vertex  5 -> Color 2
Vertex  6 -> Color 0
Vertex  7 -> Color 2
Vertex  8 -> Color 1
Vertex  9 -> Color 0
Vertex 10 -> Color 1
Vertex 11 -> Color 0
Vertex 12 -> Color 1
Vertex 13 -> Color 2
Vertex 14 -> Color 0

Validation: SUCCESS (No adjacent vertices share the same color)
```

Код програми на C++ (папка lab5/lab5_2):

https://github.com/chesswondo/distributed-systems-labs